Media Conversion Facade

Assignment 4 | Software Design Patterns
Zhuman Mukhammed | SE-2522
18 October 2026 | Java 17 | Option A

Introduction

The project demonstrates the Facade pattern using a media conversion subsystem. A caller supplies a source path, a target encoding and a destination path. One operation produces a mono output file while the facade coordinates parsing, codec selection, decoding, channel mixing, encoding and writing.

Without the facade, a normal application client would need to know the subsystem types and the valid order of their operations. This design puts that orchestration in MediaConversionFacade and keeps each algorithm in its own subsystem class. Main depends only on the facade and its public error type.

Conversion model

The assignment permits simulated media. This implementation uses two custom text encodings of actual integer sample data: pcm stores sample values, and delta stores differences between consecutive values. These files are teaching containers, not standard PCM audio files, WAV files or MP3 files. No audio library is required.

Stage	Responsibility
Read	MetadataReader validates the container structure and parses fields.
Decode	CodecFactory selects a source codec; decoded audio is validated.
Mix	AudioMixer averages left and right samples for each stereo frame.
Encode	The target codec copies samples or computes their differences.
Save	MediaWriter creates the output without overwriting existing files.

A concrete transformation

Stereo frames (1000, 3000), (2000, 4000), (-1000, 1000) become mono samples 2000, 3000, 0. Delta encoding produces 2000, 1000, -3000. Decoding reconstructs 2000, 3000, 0 by cumulative addition. The sample rate is preserved.

The mono encoding round trip is lossless. Stereo mixing intentionally loses channel information, and integer division truncates fractional averages toward zero.

UML class diagram

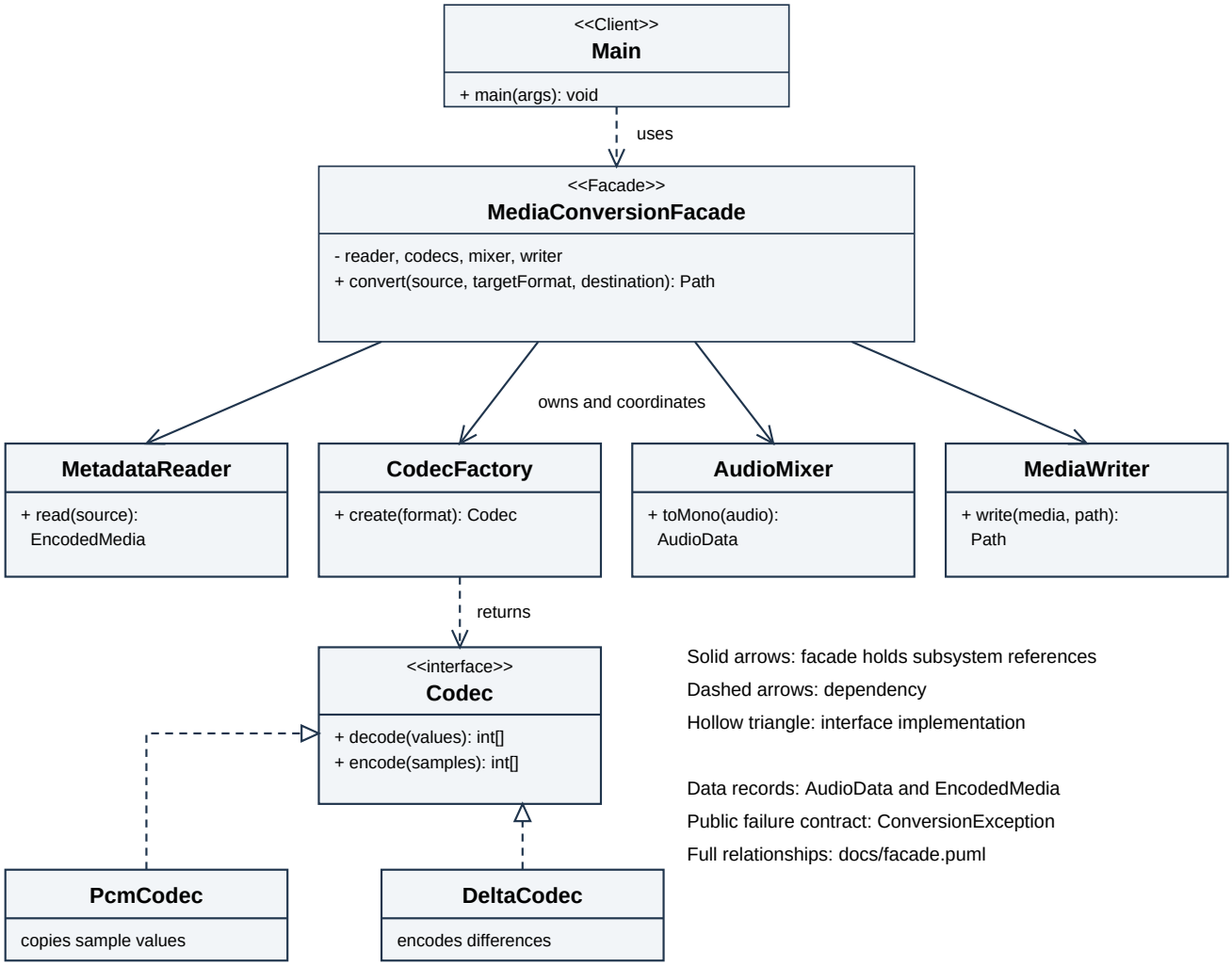

Figure 1. Client, facade and operational subsystem classes. The editable PlantUML file also shows data-record dependencies, codec creation and the exception contract.

Public API and dependency direction

```
Path convert(Path source, String targetFormat, Path destination)
        throws ConversionException
```

The method returns the written file path. Parameters and results use standard Java types; clients do not construct codecs or exchange internal audio records. The public ConversionException preserves its original cause for diagnostics.

The facade holds four subsystem objects and uses codecs returned by CodecFactory. PcmCodec and DeltaCodec implement the same Codec interface. AudioData validates decoded sample ranges and complete frames. Both data records copy arrays when constructed and when accessed.

Subsystem classes are public and never reference the facade. Advanced clients may call them directly. This preserves the defining Facade property: the simplified entry point is a convenience, not an access barrier.

Clean Code principles

The following excerpts come from the implementation. The accompanying explanations identify the purpose of each design choice.

1 Meaningful names

```
MediaConversionFacade converter = new MediaConversionFacade();
Path result = converter.convert(
        Path.of(args[0]), args[1], Path.of(args[2]));
```

Anchor: src/Main.java. The class and method state the use case. Inside the facade, source, targetFormat, destination, decoded and mono identify the role and stage of each value. A caller can understand the operation without reading codec code.

2 Single responsibility

```
AudioData mono = mixer.toMono(decoded);
EncodedMedia output = new EncodedMedia(targetFormat,
        mono.sampleRate(), mono.channels(),
        encoder.encode(mono.samples()));
return writer.write(output, destination);
```

Anchor: MediaConversionFacade.convert. The facade orders operations and passes results between them. It delegates mixing to AudioMixer, encoding to Codec and output handling to MediaWriter. A change to delta arithmetic therefore belongs in DeltaCodec, rather than in the client or facade.

3 Encapsulation at the API boundary

```
public Path convert(Path source, String targetFormat,
        Path destination) throws ConversionException
```

Anchor: MediaConversionFacade. The public conversion method contains no Codec, AudioData or EncodedMedia parameter. A simple caller uses paths and a format name. Internal records defensively copy arrays, preventing callers from silently mutating stored data through a shared array reference.

The facade does not make subsystems private or inaccessible. It reduces the dependencies required for an ordinary client while preserving direct access for specialized clients.

Clean Code and failure handling

4 Small focused operations

```
for (int frame = 0; frame < mono.length; frame++) {
    int left = stereo[frame * 2];
    int right = stereo[frame * 2 + 1];
    mono[frame] = (left + right) / 2;
}
```

Anchor: AudioMixer.toMono. This loop handles only channel mixing. It does not parse files, select formats or write output. AudioData has already checked that stereo frames are complete and that samples fit signed 16-bit range, so adding two samples is safe within a Java int.

5 Explicit error handling and cleanup

```
} catch (IOException | IllegalArgumentException exception) {
    throw new ConversionException(
            "Conversion failed: " + exception.getMessage(),
            exception);
}
```

Anchor: MediaConversionFacade.convert. The client receives one checked exception for expected conversion failures. The original exception remains available as the cause. Main prints the diagnostic and exits with status 1; incorrect command usage exits with status 2.

```
try {
    Files.writeString(temporary, content, StandardCharsets.UTF_8);
    return Files.move(temporary, target);
} finally {
    Files.deleteIfExists(temporary);
}
```

Anchor: MediaWriter.write. Validation and transformations finish before the writer runs. An existing destination is rejected. Output is first written to a temporary sibling file, then moved without a replace option. The finally block removes a remaining temporary file on success or failure. This avoids deliberately overwriting user data and publishing a partially written file during ordinary handled failures.

Design tradeoffs

The facade constructs its fixed collaborators directly to keep this small assignment easy to follow. Constructor injection would help a larger application substitute services. CodecFactory uses a two-case switch; adding a third codec requires updating that factory. Files are loaded into memory, so this design targets small examples. It does not claim crash-safe durability, standard audio compression, playback or resampling.

Verification and demonstration

The verification suite calls the facade and checks output file contents. It covers 25 successful and failing scenarios, including sample-level round trips, boundary values, malformed input, invalid metadata, unsupported formats, null arguments and protection of existing files. The saved verification log records the compiler and runtime used.

Coverage	Observed behavior
Conversion	Expected delta values, decoded PCM, mono round trip, same-format conversion, rounding, stereo delta input.
Invalid input	Unknown source/target format, missing file, bad header, non-numeric data, incomplete frames, invalid rate/channels, sample overflow, empty audio/token.
Output and API	Missing destination directory, preserved output/source, null arguments, no temporary files left.

```
javac --release 17 -encoding UTF-8 -Xlint:all -Werror \
    -d out @sources.txt tests/FacadeTests.java
java -cp out FacadeTests

java -cp out Main samples/stereo.pcm delta converted.delta
java -cp out Main converted.delta pcm restored.pcm
```

The compiler command above is shown across two lines for readability; run it on one line. In PowerShell, quote the argument-file parameter as `'@sources.txt'`. The provided build.ps1 automates compilation and verification. Use new destination filenames for repeated demonstrations.

Conclusion

MediaConversionFacade gives the client one meaningful operation over a collaborating subsystem. It hides the order of conversion steps, codec selection and output coordination. The subsystem retains the actual algorithms and remains directly usable, independent of the facade. The implementation demonstrates simplification through coordination rather than moving all behavior into one large class.

Repository and submission

GitHub repository: not yet published. The actual repository URL must be inserted before submission. The local project includes incremental Git commits, source files, examples, tests, the UML source and defense notes.

Assignment source: Assignment_4_Facade_Pattern_Instructions.docx, supplied course handout. The handout limits generative AI to examples and clarification. The student should independently review and adapt this worked example before submitting under that policy.